

Adaptalyfe Flutter — Android & iOS Documentation

Overview

The Adaptalyfe mobile application will be built using:

Layer	Technology
Mobile Framework	Flutter
Language	Dart
State Management	BLoC
Backend	Existing Node.js + Express API
Database	Existing PostgreSQL
Authentication	Existing session-token API
Platform	Android + iOS
JSON Serialization	json_serializable / manual models
Local Storage	FlutterSecureStorage
Architecture	Feature-based Clean Architecture
HTTP Client	Dio
Navigation	GoRouter
Dependency Injection	GetIt
API	REST API

Important

The Flutter application will use the **existing backend and API flow**.

No new authentication backend is required.

Production API:

<https://app.getadaptalyfeapp.com>

The existing backend uses:

POST /api/login

POST /api/register

POST /api/logout

GET /api/user

The existing login flow validates credentials, creates a session, and returns a session token and user information. The frontend stores the token for subsequent authenticated requests.

Application Flow

App Launch



Splash Screen



Check Stored Session Token



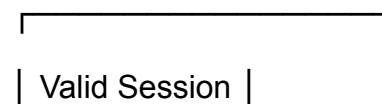
↓ Yes

↓ No



GET /api/user

Login Screen



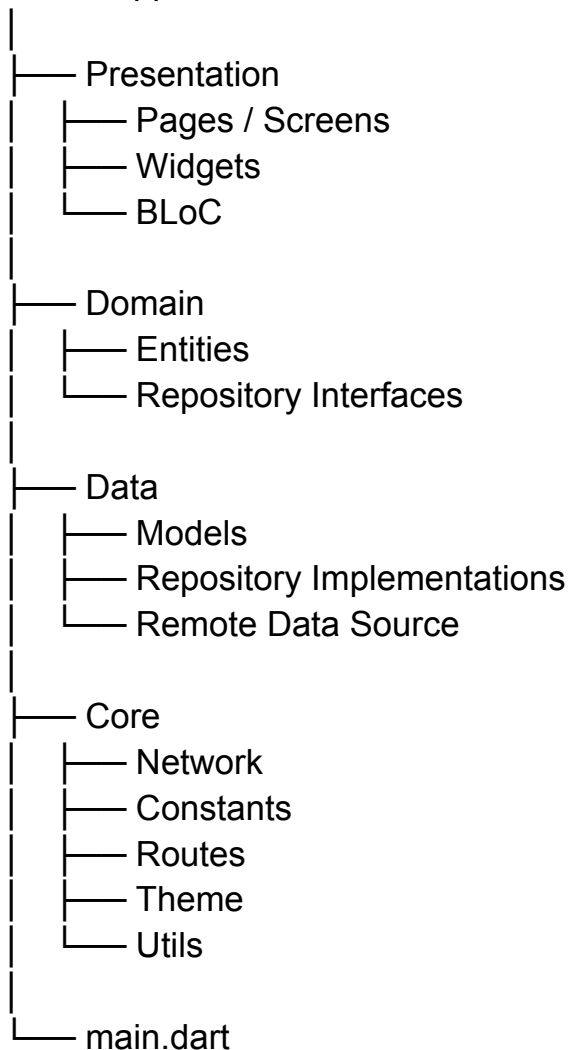


Home Screen

The existing web application follows the same basic principle: it reads the stored session token and checks `/api/user` before allowing access to protected screens.

Architecture :-

Flutter App



Splash Screen

Purpose

The Splash Screen is the first screen displayed when the mobile application starts.

Responsibilities

- Display application logo
- Initialize application
- Check locally stored session token
- Validate existing session
- Navigate to Login or Home

Flow

- SplashScreen
- ↓
- SplashBloc
- ↓
- Read session token
- ↓
- Token available?
- ↓
- GET /api/user
- ↓
- Valid?
- ┌──────────┴──────────┐
- Yes No
- ↓ ↓
- Home Login

BLoC

Event

SplashStarted

States

- SplashInitial
 - SplashLoading
 - SplashAuthenticated
 - SplashUnauthenticated
- SplashError

BLoC Responsibility

- SplashBloc
- ↓
- LocalStorage
- ↓
- Session Token
- ↓
- AuthRepository
- ↓
- GET /api/user

The existing backend's `/api/user` endpoint is the source used to determine whether the current session is authenticated.

Login Screen

Purpose

Allows an existing user to log into the application.

UI

Logo

Welcome Back

Username

Password

[LOGIN]

Forgot Password?

Don't have an account?

Sign Up

API

POST `/api/login`

Request

```
{
  "username": "user@example.com",
  "password": "*****"
}
```

Response

Existing backend returns:

```
{
  "sessionToken": "...",
  "user": {}
}
```

The existing login implementation creates the server session and returns a session token and user data.

BLoC Flow

User taps Login



LoginSubmitted



AuthBloc



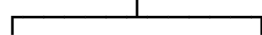
AuthRepository



POST /api/login



Success?



Yes

No



Save Token



AuthError



Home Screen

Events

LoginSubmitted

LogoutRequested

CheckAuthentication

States

AuthInitial

AuthLoading

Authenticated

Unauthenticated

AuthError

Signup Screen

Purpose

Allows a new user to create an Adaptalyfe account.

UI

Create Account

Name

Username
[]

Password
[]

Confirm Password
[]

[SIGN UP]

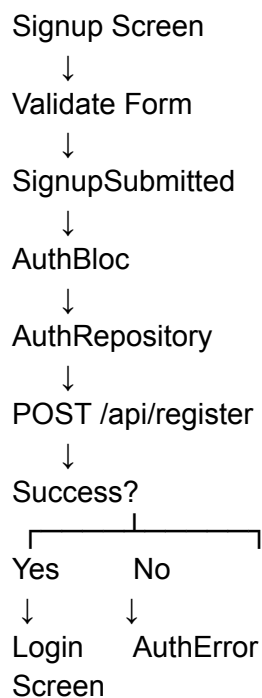
Already have an account?
Login

API

POST /api/register

The existing backend contains the registration endpoint in the authentication section alongside login and logout.

Flow



Important

Flutter should follow the **existing backend registration request/response format**. Do not create a new registration API.

Home Screen

Purpose

Home is the main authenticated screen after successful login.

The existing application dashboard is the main logged-in view and provides access to the application's modules.

Initial Home Flow

HomeScreen



HomeBloc



GET /api/user



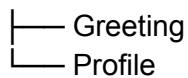
Load user information



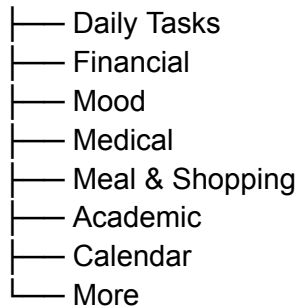
Display Dashboard

Initial Home UI

Header



Dashboard



The existing application contains modules including daily tasks, financial tracking, mood tracking, medical information, meal planning, academic planning, calendar, sleep tracking, resources, rewards, and settings.

Home BLoC

Events

HomeStarted

RefreshHome

States

HomeInitial

HomeLoading

HomeLoaded

HomeError

Flow

HomeStarted
↓
HomeBloc
↓
HomeRepository
↓
GET /api/user
↓
HomeLoaded
↓
Update UI

Authentication Architecture

Authentication should be centralized.

LoginScreen
↓
AuthBloc
↓
AuthRepository
↓
ApiClient
↓
Existing Express Backend

API Client

All API requests should pass through one common API client.

ApiClient
↓
Base URL
↓
Authorization Header
↓
HTTP Request

The existing frontend sends the stored token using:

Authorization: Bearer <sessionToken>

The backend then validates the session and populates the authenticated user session.

Local Storage

Flutter should persist the existing session token locally.

Login Success



sessionToken



LocalStorage

On application restart:

Splash



LocalStorage



sessionToken



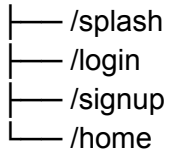
GET /api/user

The current mobile implementation also persists the session token across app restarts.

Navigation

Recommended routes:

/



Navigation rules:

App Start



Splash

Not Authenticated



Login

New User



Signup

Authenticated

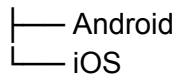


Home

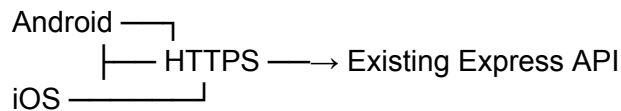
Android & iOS

The same Flutter codebase will support:

Flutter



Backend remains common:



Production API:

<https://app.getadaptalyfeapp.com>

The existing mobile architecture uses Android and iOS native projects while communicating with the same production backend.

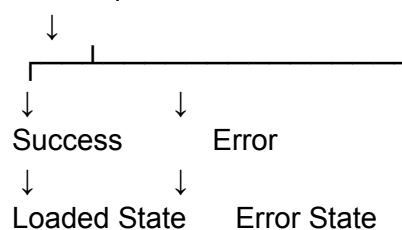
Error Handling

All screens should handle:

Loading
Success
Error
Empty

Example:

API Request



Common errors:

- Invalid username/password
- Network error
- Server error
- Session expired
- Invalid response
- Timeout

Logout Flow

Home
↓
Logout
↓
AuthBloc
↓
POST /api/logout
↓
Clear local session token
↓
Login Screen

The existing backend already provides `/api/logout`, and the current frontend clears the stored token during logout.

First Development Phase

Only these screens should be implemented first:

1. Splash Screen
2. Login Screen
3. Signup Screen
4. Home Screen

Implementation order

Step 1
Flutter project setup
↓
Step 2
API Client
↓
Step 3
Local Storage
↓
Step 4
User Model
↓
Step 5
Auth Repository
↓
Step 6
Auth BLoC
↓
Step 7

Splash Screen



Step 8

Login Screen



Step 9

Signup Screen



Step 10

Home Screen



Step 11

Android testing



Step 12

iOS testing

Important Architecture Rule

Do not put API calls directly inside screens.

✗ Avoid

LoginScreen



HTTP Request

✓ Use

LoginScreen



AuthBloc



AuthRepository



ApiClient



Existing Backend

This keeps the Flutter application clean, testable, and scalable for adding the remaining Adaptalyfe modules later.